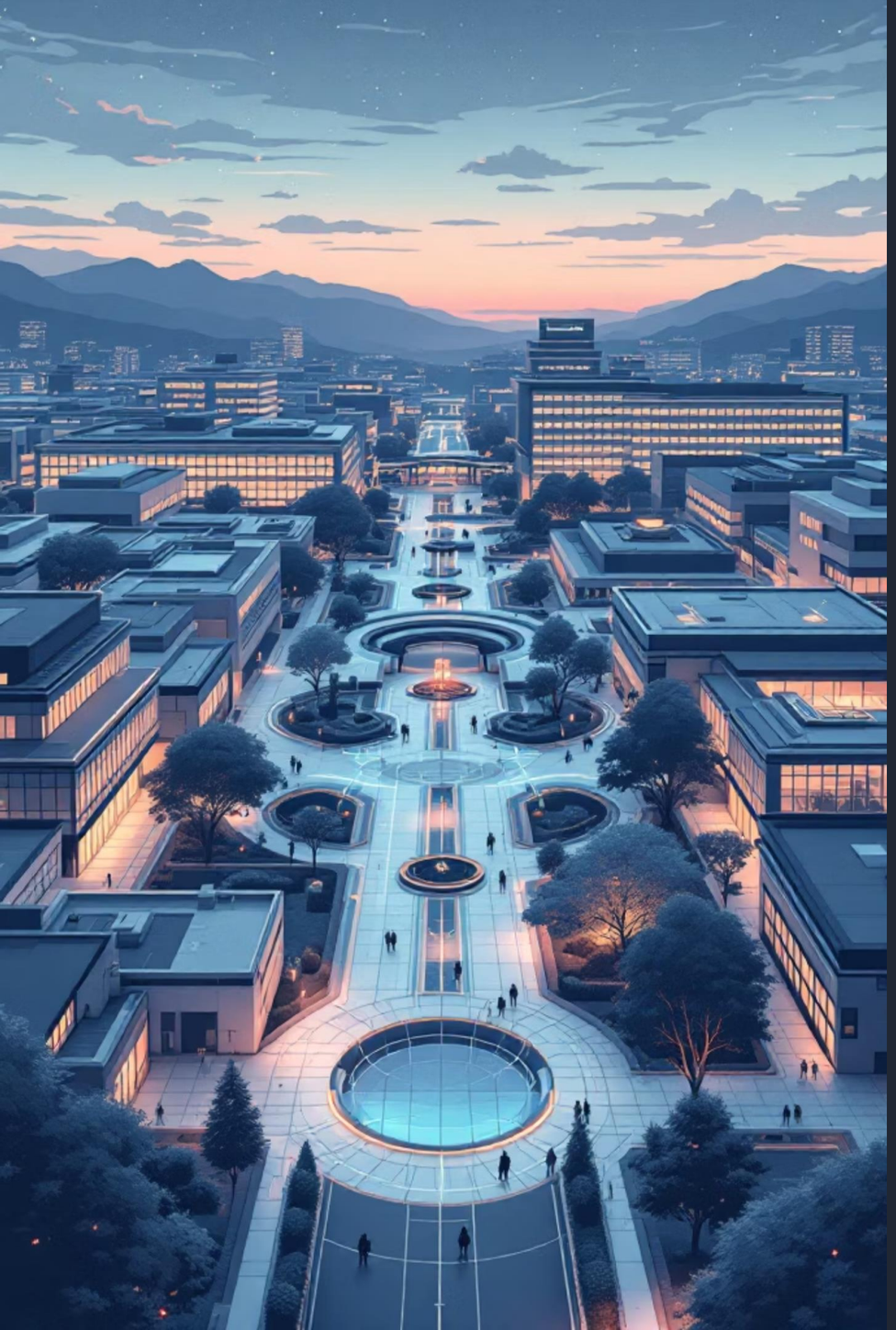


FindIt: Smart Lost-and-Found

Automated Campus Web Application

Team: Sparsh Kumar (S24CSEU0428)
Akshat Madan (S24CSEU0294)
Harsh Raj (S24CSEU0765)
Samagra Gulati (S24CSEU0451)

University Name: Bennett University
Mentor: Prof. Priyanka Gupta



Fragmented Campus Recovery

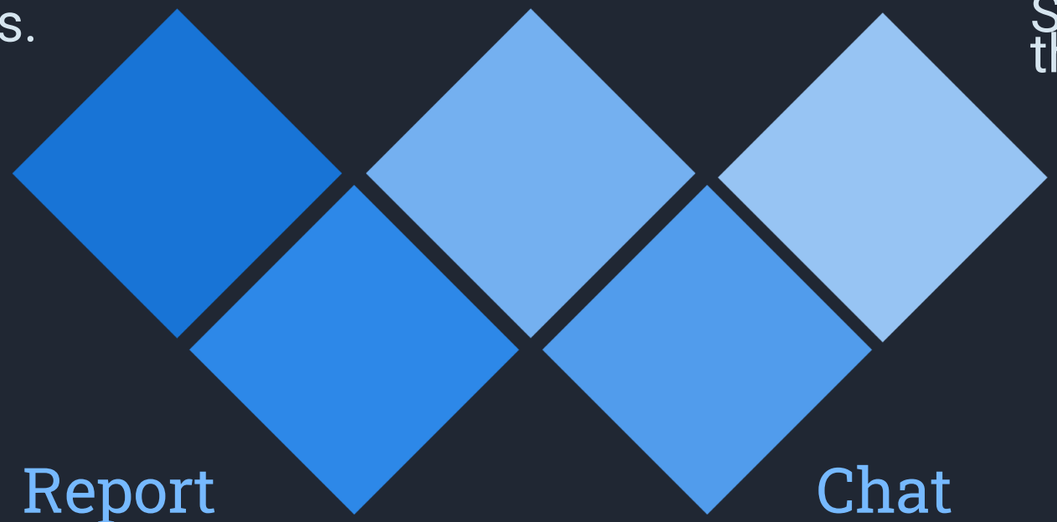
- Problem: Communication fails across campus groups.
- Manual: Searching wastes valuable student time.
- Clutter: Unclaimed items pile up quickly.
- Solution: FindIt centralizes the recovery process.
- Smart: AI algorithms match items instantly.

Automated Recovery Lifecycle

Verify
Answer secret item security questions.

Match
AI pairs similar item reports.

Recover
Securely handoff the confirmed item.



Report
Submit details and item photos.

Chat
Direct messaging between both users.

Interactive User Experience



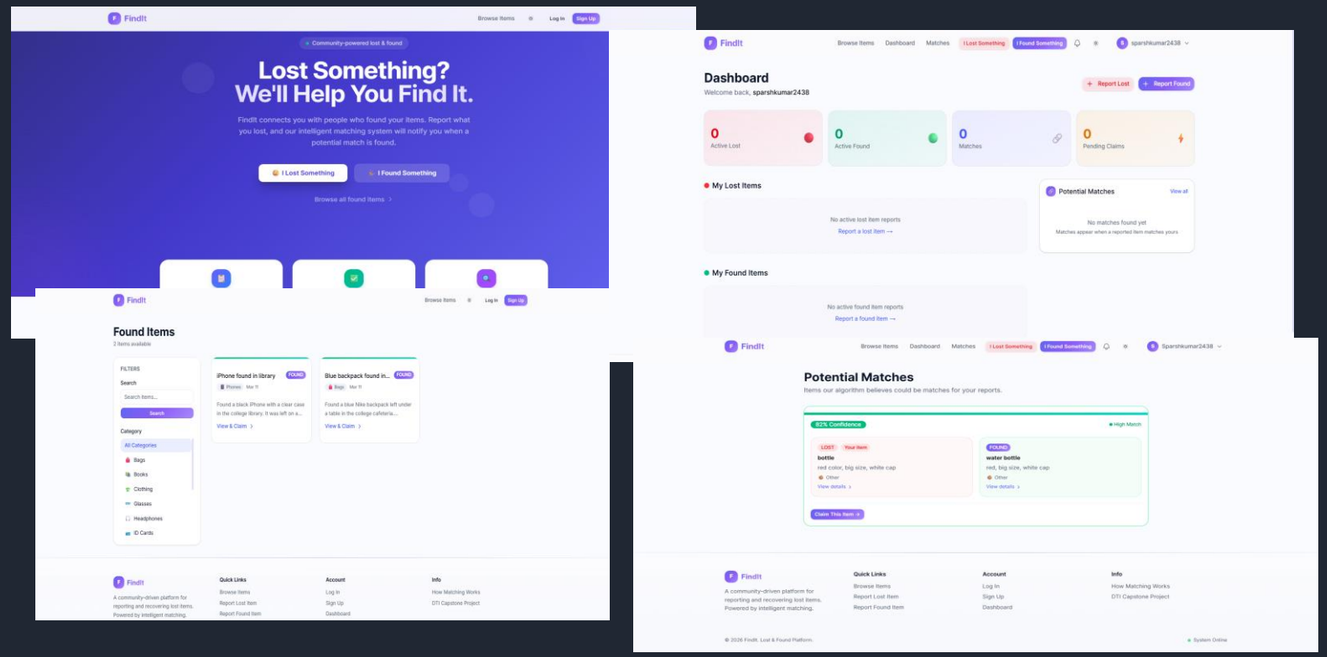
- Visuals: Seamlessly attach clear item photos.
- Dashboard: Manage active reports and claims.
- Messaging: Chat directly with item finders.
- Gallery: Browse or search found items.

Real-Time Notifications

- Alerts: Get notified of AI matches.
- Updates: Keep users engaged without refreshing.
- Speed: Drive immediate claim and recovery.
- Status: Monitor match and claim progress.

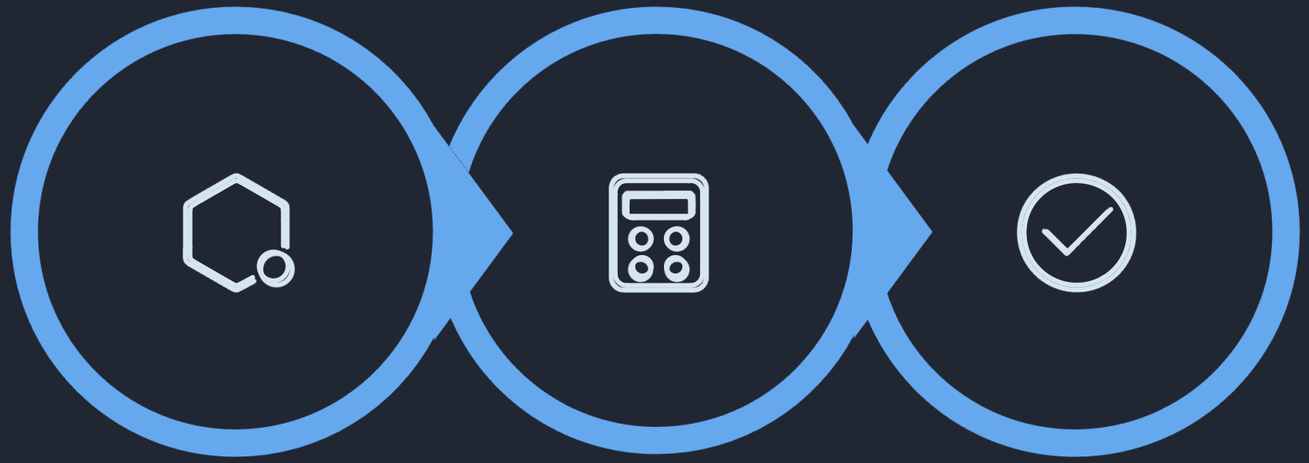


Scalable Full-Stack Architecture



- Frontend: Next.js 16 and React 19.
- Styling: Tailwind CSS 4 and shadcn/ui.
- Backend: Next.js API routes power logic
- Database: PostgreSQL managed by Drizzle ORM.
- Cloud: Secure backend hosted on Supabase.

Database & Data Flow



- Users: Secure accounts and user authentication.
- Reports: Stores items and ML embeddings.
- Matches: Links automated lost-found pairings.
- Claims: Tracks ownership verification and status.
- Messages: Stores direct user chat conversations.

- Tokenize
- Compute IoU
- Score ≥80%

ML Semantic Text Matching

- Engine: Powered by advanced transformer models.
- Embeddings: Converts item descriptions into vectors.
- Semantic: Understands deep contextual meaning accurately.
- Beyond: Catches matches basic keywords miss.
- Automation: Instantly pairs highly similar reports.



Secure Verification Flow

- Security: Claimants answer custom verification questions.
- Review: Finders manually approve claim requests.
- Authentication: Secured by NextAuth version 5.
- Protection: Rigorous bcryptjs password hashing system.
- Validation: React Hook Form and Zod.



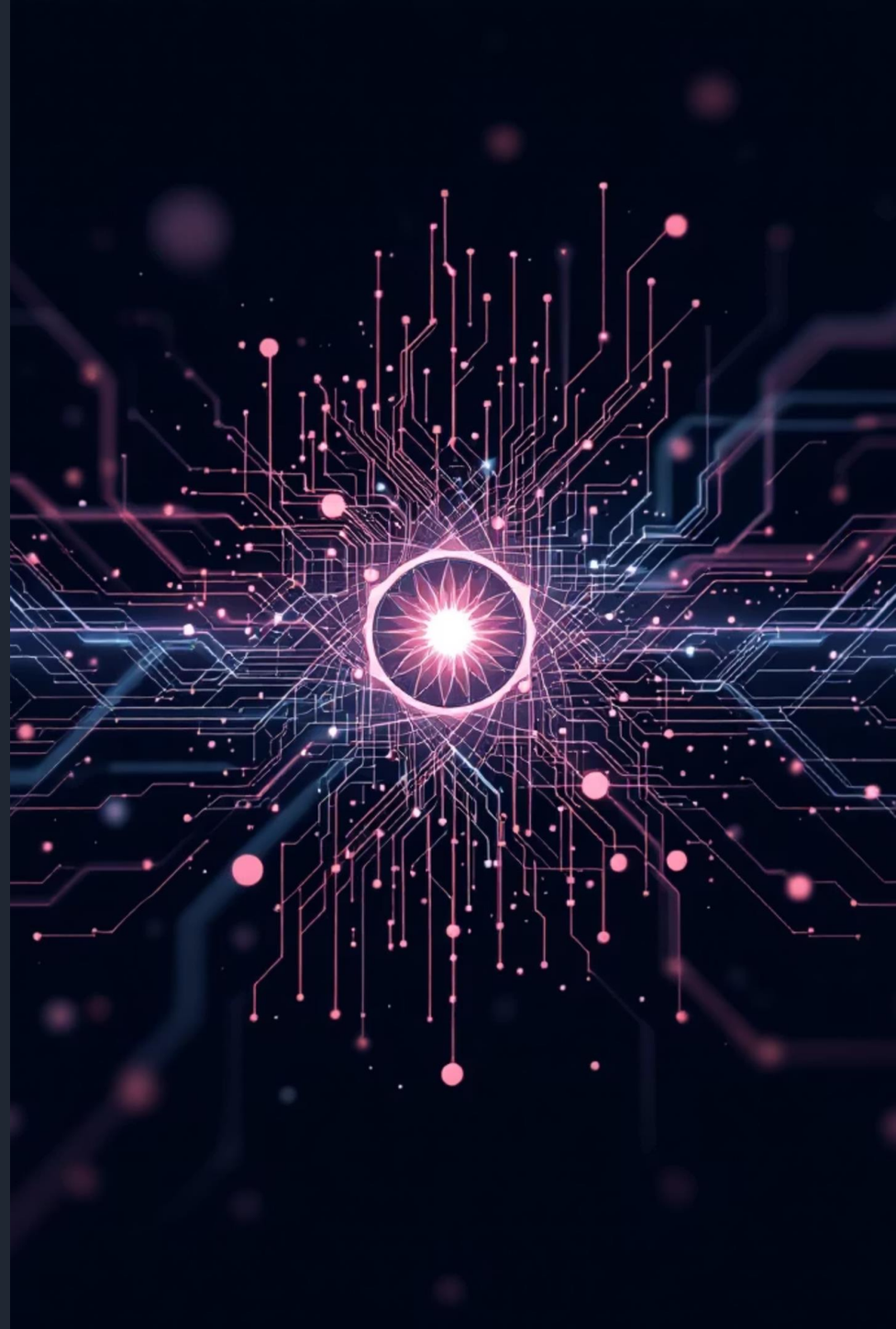
Navigating Technical Hurdles

Key Learnings Acquired:

- Mastering modern frameworks via Next.js
- Modeling relational databases using Drizzle ORM
- Implementing robust session security flows

Critical Challenges Overcome:

- Building accurate similarity engine from scratch
- Managing cascading real-time database relationships
- Engineering zero-latency match computations



Project Team Overview

Sparsh Kumar (Project Lead)

- Architected Next.js foundation
- Designed database schema
- Implemented Deep learning transformer model text embeddings.

Akshat Madan (Backend Developer)

- Built NextAuth security system
- Developed custom logic
- Ensured strict Zod validation

Harsh Raj (Frontend Developer)

- Integrated Shadcn/ui components
- Optimized responsive page experience
- Developed dynamic user dashboards

Samagra Gulati (DevOps & Testing)

- Engineered query engine
- High-level testing protocols
- Managed application deployment

